

Comparación de algoritmos de aprendizaje automático distribuido para la predicción de propina baja en viajes de taxi de Nueva York con Apache Spark MLlib

Niels Pacheco*

*Escuela de Posgrado, Universidad Nacional de Ingeniería

Maestría en Inteligencia Artificial — Curso de Big Data

Lima, Perú

E-mail: nielspacheco1997@gmail.com

Resumen—El crecimiento de los registros operativos del transporte urbano plantea el reto de aplicar aprendizaje automático sobre volúmenes que exceden la capacidad de una sola máquina con herramientas tradicionales. Este trabajo formula una pregunta de investigación de naturaleza comparativa: ¿qué algoritmo de clasificación distribuida ofrece el mejor compromiso entre desempeño predictivo y costo computacional para predecir, sobre decenas de millones de viajes de taxi de Nueva York, si un viaje pagado con tarjeta dejará una propina baja? Utilizando seis meses de los registros públicos de la NYC Taxi and Limousine Commission (19,5 millones de viajes crudos, 14,9 millones tras depuración) y la biblioteca Apache Spark MLlib, se comparan cuatro clasificadores (regresión logística, árbol de decisión, random forest y gradient-boosted trees) bajo un fuerte desbalance de clases ($\approx 9,7:1$), todos con ponderación de clases. Los resultados muestran que la *accuracy* es engañosa en este escenario: un clasificador trivial alcanzaría un 90,6 % de exactitud con recall nulo sobre la clase de interés, mientras que los modelos ponderados recuperan entre el 50 % y el 64 % de los viajes de propina baja. El gradient-boosted trees logra el mejor desempeño predictivo ($AUC = 0,686$ y recall de la minoría = 0,64) pero al mayor costo de cómputo; el random forest ofrece un AUC casi equivalente (0,680) en la mitad del tiempo, y la regresión logística es la alternativa más eficiente. Adicionalmente, dos experimentos de escalabilidad evidencian un crecimiento sublineal del tiempo de entrenamiento con el volumen ($20\times$ datos $\rightarrow 5\times$ tiempo) y un *speedup* de $4,71\times$ al pasar de 1 a 8 núcleos, consistente con la ley de Amdahl. Se concluye que, bajo desbalance, la elección del algoritmo debe guiarse por el AUC y el recall de la minoría, sopesando el costo computacional, y no por la exactitud.

Index Terms—Big Data; Apache Spark; MLlib; aprendizaje distribuido; desbalance de clases; escalabilidad; NYC Taxi.

I. INTRODUCCIÓN

La generación masiva de datos por sistemas de transporte, sensores y plataformas digitales ha consolidado el paradigma de Big Data, caracterizado por las llamadas cinco V: volumen, velocidad, variedad, veracidad y valor [1]. Cuando el volumen de los datos supera la memoria y el cómputo de un único nodo, distribuir el procesamiento se vuelve inevitable. Apache Spark se ha establecido como el motor de referencia para este tipo

de cargas gracias a su modelo de cómputo en memoria y a su biblioteca de aprendizaje automático escalable, MLlib [2], [3].

Los registros de viajes de taxi de la ciudad de Nueva York constituyen un caso de estudio representativo: cada mes se publican millones de viajes con información espacial, temporal y económica. Sobre ellos es posible plantear problemas de valor directo para el negocio del transporte, como anticipar la propina que recibirá un conductor. Para quien maneja el taxi, la propina representa una fracción apreciable de su ingreso; poder anticiparla ayudaría a diseñar incentivos, a organizar mejor el despacho y a entender el comportamiento del pasajero.

Este trabajo aborda una tarea de clasificación binaria con relevancia práctica y con una propiedad estadística que el curso enfatizó reiteradamente: el desbalance de clases. Se define como clase positiva el viaje con *propina baja* (inferior al 10 % de la tarifa), que resulta minoritario. Más que construir un clasificador puntual, el trabajo compara de forma controlada cuatro algoritmos distribuidos y mide cómo escala el sistema, recorriendo el ciclo de vida completo del dato (ingesta, procesamiento distribuido, modelado y visualización) estudiado a lo largo del curso.

II. ESTADO DEL ARTE Y TRABAJOS RELACIONADOS

El procesamiento distribuido de datos masivos tiene su antecedente en el modelo MapReduce [4], posteriormente superado en eficiencia por Apache Spark, que evita la materialización en disco entre etapas y unifica el procesamiento por lotes, en flujo y de aprendizaje automático [2]. MLlib aporta implementaciones distribuidas de los algoritmos clásicos de clasificación, regresión y *clustering* sobre la API de DataFrames y *pipelines* [3]. Estudios comparativos de herramientas de aprendizaje sobre el ecosistema Hadoop sitúan a Spark MLlib como una de las opciones más maduras para aprendizaje a gran escala [5].

El aprendizaje con clases desbalanceadas es un problema ampliamente estudiado [6]. Las estrategias habituales

incluyen el remuestreo sintético de la clase minoritaria mediante SMOTE [7], el aprendizaje sensible al costo [8] y la ponderación de instancias. En el contexto de Big Data, el remuestreo introduce retos adicionales de escalabilidad, pues altera el volumen de datos que debe distribuirse y procesarse [9]. Por ello, en este trabajo se prefiere la ponderación de clases, que no incrementa el tamaño del conjunto.

Entre los algoritmos comparados, random forest [10] y el *gradient boosting* [11], popularizado a gran escala por XGBoost [12], representan dos familias de ensambles con compromisos distintos entre sesgo, varianza y costo. La evaluación bajo desbalance exige métricas robustas; el análisis ROC y el área bajo la curva (AUC) son el estándar al ser insensibles a la proporción de clases [13]. Finalmente, trabajos previos sobre los datos de taxis de Nueva York han abordado el análisis de demanda y la predicción de duración de viajes con enfoques de Big Data y *boosting* [14], [15], aunque la predicción de propina baja como problema de clasificación desbalanceada ha recibido menos atención.

III. PROBLEMA DE INVESTIGACIÓN

Se plantea la siguiente pregunta de investigación:

Sobre decenas de millones de viajes de taxi de Nueva York, ¿qué clasificador distribuido de Spark MLlib ofrece el mejor compromiso entre desempeño predictivo (medido con métricas robustas al desbalance) y costo computacional, para predecir si un viaje pagado con tarjeta dejará una propina baja?

De ella se derivan tres preguntas específicas: (i) ¿cómo se comportan regresión logística, árbol de decisión, random forest y GBT en AUC, F1 y recall de la clase minoritaria una vez aplicado el manejo de desbalance?; (ii) ¿cuál es el costo de entrenamiento de cada uno?; y (iii) ¿cómo escala el tiempo de cómputo al aumentar el volumen de datos y el número de núcleos del procesamiento distribuido?

IV. METODOLOGÍA

La solución sigue el ciclo de vida del dato resumido en la Fig. 1. Cada etapa se asocia a la dimensión de Big Data que aborda.

IV-A. Infraestructura

Los experimentos se ejecutaron con Apache Spark 3.5.3 sobre Java 11 y Python 3.11, en modo local (`local[*]`) en una máquina con 16 núcleos y 68 GB de RAM física, asignando 24 GB a la memoria del *driver* de Spark. El modo local instancia un *executor* multihilo que simula el paralelismo de un clúster, lo que garantiza reproducibilidad sin costos de nube; el mismo código migra sin cambios a un clúster gestionado como AWS EMR o Databricks.

IV-B. Conjunto de datos y depuración

Se emplearon los registros públicos de *Yellow Taxi* de la NYC Taxi and Limousine Commission correspondientes a los seis primeros meses de 2023, en formato Apache Parquet. Dado que el esquema físico de algunas columnas varía entre

meses (p.ej. `passenger_count` como entero o real), la ingesta lee cada archivo y castea las columnas a tipos canónicos antes de unirlos, evitando errores de fusión de esquemas. La depuración descartó registros con valores no plausibles: solo viajes pagados con tarjeta de crédito (la propina en efectivo no se registra), con tarifa y distancia positivas y acotadas, entre uno y seis pasajeros, duración entre 1 y 180 minutos y velocidad media menor a 100 mph. De los 19,5 millones de viajes crudos se conservaron 14,9 millones.

IV-C. Definición de la etiqueta y variables

La etiqueta binaria marca como positivo (1) el viaje con *propina baja*, definida como $\text{tip_amount}/\text{fare_amount} < 0,10$. Para evitar *fuga de objetivo* se excluyeron explícitamente como predictores la propina y el importe total (que la contiene por construcción). El vector de variables combina catorce atributos numéricos (distancia, número de pasajeros, duración, velocidad media, tarifa, recargos, hora del día, indicadores de fin de semana y de madrugada, y las zonas de origen y destino) con dos atributos categóricos, el día de la semana y el código de tarifa, codificados con *one-hot*. La construcción del vector reproduce el patrón canónico de MLlib: `StringIndexer`, `OneHotEncoder` y `VectorAssembler` encadenados en un `Pipeline`.

IV-D. Manejo del desbalance

La clase minoritaria representa el 9,36 % de los viajes, un desbalance de 9,69:1 (Fig. 3). Sin tratamiento, un clasificador trivial que prediga siempre “propina normal” alcanzaría un 90,6 % de exactitud y 0 % de recall sobre la clase de interés. Se aplicó ponderación de instancias: a cada viaje de la clase c se le asigna un peso $w_c = N/(2N_c)$, de modo que ambas clases contribuyen por igual a la función de pérdida. Esta técnica, soportada nativamente por los cuatro clasificadores en Spark 3.x mediante el parámetro `weightCol`, garantiza una comparación equitativa y no incrementa el volumen de datos, a diferencia del remuestreo.

IV-E. Modelos y evaluación

Se compararon cuatro clasificadores de `pyspark.ml`: regresión logística (línea base lineal), árbol de decisión (profundidad 10), random forest (50 árboles, profundidad 10) y gradient-boosted trees (50 iteraciones, profundidad 6). La comparación se realizó sobre una muestra de dos millones de viajes dividida en 80 % para entrenamiento (1,60 M) y 20 % para prueba (0,40 M), idéntica para todos los modelos. Se reportan el AUC-ROC mediante `BinaryClassificationEvaluator`, y la exactitud, precisión, recall y F1 ponderados mediante `MulticlassClassificationEvaluator`, además del recall de la clase minoritaria calculado explícitamente.

IV-F. Experimentos de escalabilidad

Se diseñaron dos benchmarks. En la *escalabilidad de datos* se fijó el paralelismo y se entrenó la regresión logística sobre fracciones crecientes (de 0,75 M a 14,9 M de filas), midiendo

Ciclo de vida del dato — Predicción de propina baja (NYC Taxi, Spark MLlib)

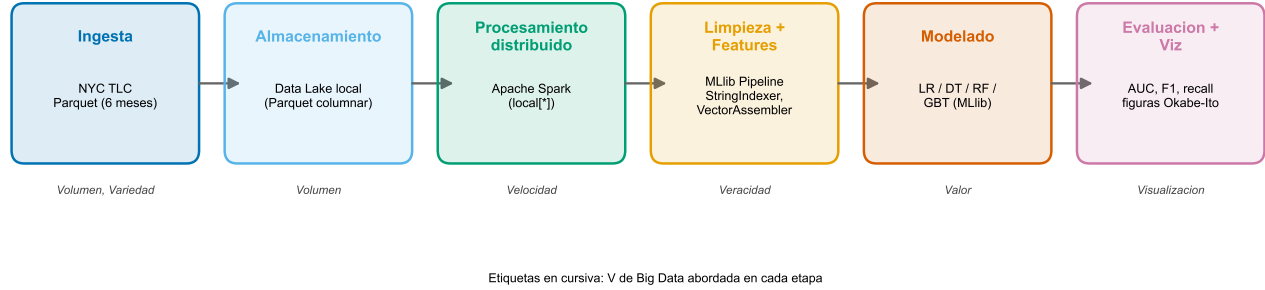


Figura 1. Arquitectura del ciclo de vida del dato. La ingesta normaliza el esquema de los Parquet mensuales; Spark realiza el procesamiento distribuido; un *pipeline* de MLlib construye las variables y entrena los modelos; la evaluación reporta métricas robustas al desbalance y produce las visualizaciones.

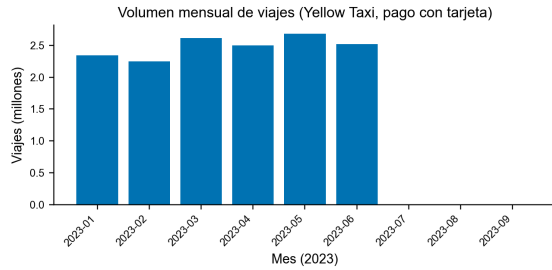


Figura 2. Volumen mensual de viajes pagados con tarjeta (2023). El sistema procesa del orden de 2,5 millones de registros por mes.

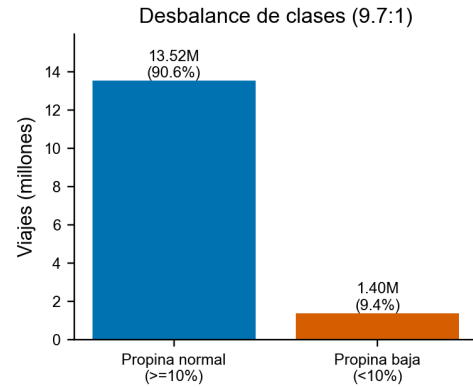


Figura 3. Desbalance de clases: la propina baja es minoritaria (9,4 %), con una razón de 9,7:1 frente a la clase mayoritaria.

el tiempo de ajuste. En la *escalabilidad fuerte* se fijó el volumen en tres millones de viajes y se varió el número de núcleos del *executor* (1, 2, 4 y 8), midiendo el *speedup* respecto a un núcleo. En ambos casos los datos se materializaron en caché antes de cronometrar para no contabilizar la lectura perezosa.

V. EXPERIMENTOS Y RESULTADOS

V-A. Análisis exploratorio

El volumen mensual se mantiene estable en torno a 2,5 millones de viajes (Fig. 2), confirmando la magnitud del problema. La tasa de propina baja presenta un patrón temporal claro, con mayor incidencia en la madrugada (donde alcanza $\approx 17,8$ % frente al $\approx 9,4$ % global), lo que sugiere que las variables temporales aportan señal. No obstante, el considerable solapamiento de las distribuciones de las variables operativas entre ambas clases anticipa que la tarea es intrínsecamente difícil.

V-B. Comparación de modelos

La Tabla I y la Fig. 4 resumen el desempeño de los cuatro modelos entrenados con ponderación de clases. Antes de compararlos conviene fijar una referencia: un clasificador trivial que prediga siempre la clase mayoritaria (“propina normal”) alcanzaría una exactitud de 90,6 % pero un recall nulo sobre la clase de interés. Este contraste es la primera evidencia de que la exactitud es una métrica engañosa bajo desbalance: tras la ponderación, la exactitud de los modelos

Cuadro I

DESEMPEÑO EN EL CONJUNTO DE PRUEBA (0,40 M DE VIAJES), CON PONDERACIÓN DE CLASES EN LOS CUATRO MODELOS. RECALL+ ES EL RECALL DE LA CLASE MINORITARIA (PROPINA BAJA). LA EXACTITUD (ACC.) SE REPORTA POR COMPLETITUD, NO COMO CRITERIO DE SELECCIÓN. EN NEGRITA, EL MEJOR VALOR DE CADA COLUMNA SALVO ACC.

Modelo	AUC	F1	Acc.	Recall+	Tiempo (s)
Regresión logística	0,656	0,753	0,691	0,501	9,0
Árbol de decisión	0,541	0,690	0,609	0,637	3,1
Random forest	0,680	0,720	0,646	0,598	27,6
GBT	0,686	0,702	0,623	0,637	49,2

desciende al rango 0,61–0,69 mientras que su recall de la minoría asciende a 0,50–0,64, lo que indica que ahora sí detectan los viajes de propina baja en lugar de ignorarlos.

El gradient-boosted trees obtiene el mejor desempeño predictivo, con el mayor AUC (0,686) y, junto al árbol, el mayor recall de la minoría (0,637), pero a costa del mayor tiempo de entrenamiento (49,2 s). El random forest alcanza un AUC prácticamente equivalente (0,680; la diferencia de 0,006 frente al GBT es despreciable con un único particionado) y un recall de 0,60 en aproximadamente la mitad del tiempo (27,6 s),

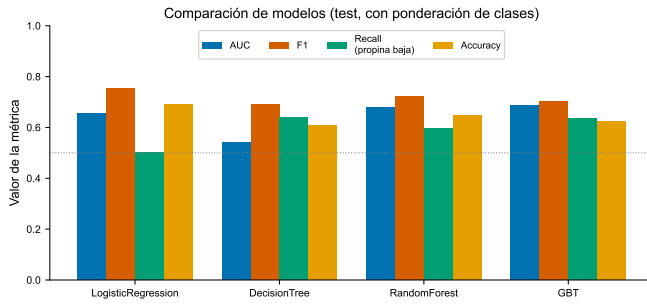


Figura 4. Comparación de modelos con ponderación de clases. Los ensambles (random forest y GBT) logran el mayor AUC; el árbol y el GBT, el mayor recall de la clase minoritaria. La exactitud no refleja la utilidad real bajo desbalance.

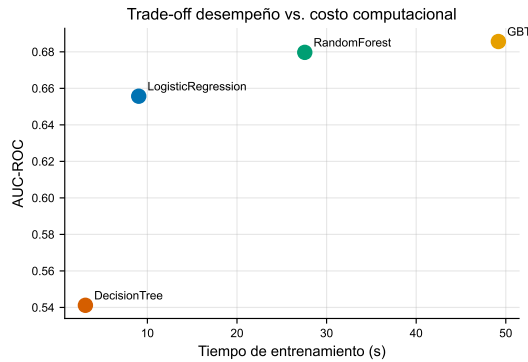


Figura 5. Compromiso entre AUC y tiempo de entrenamiento. El GBT maximiza el AUC al mayor costo; el random forest ofrece un AUC casi igual a la mitad del tiempo; la regresión logística es el punto más eficiente.

por lo que constituye un compromiso atractivo. La regresión logística es la opción más eficiente entre los modelos de AUC competitivo: cinco veces más rápida que el GBT (9,0 s) con un AUC de 0,656. El árbol de decisión es el más veloz (3,1 s) y empata en recall, pero su AUC es el más bajo (0,541), señal de una capacidad de *ranking* pobre. La Fig. 5 sitúa cada modelo en el plano desempeño–costo y hace explícito el frente de compromiso.

V-C. Escalabilidad

El tiempo de entrenamiento de la regresión logística crece de forma sublineal con el volumen: multiplicar los datos por veinte (de 0,75 M a 14,9 M) solo multiplica el tiempo por cinco (de 6,1 a 30,3 s), gracias al paralelismo de datos de Spark (Fig. 6). El experimento de escalabilidad fuerte (Fig. 7) muestra un *speedup* de $1,93\times$ con dos núcleos (próximo al ideal) que se aleja progresivamente de la recta ideal hasta $4,71\times$ con ocho núcleos. Este comportamiento sublineal es consistente con la ley de Amdahl [16]: la fracción serial del trabajo y los costos de coordinación limitan la ganancia marginal de cada núcleo adicional.

VI. DISCUSIÓN

Los resultados permiten responder la pregunta de investigación, y la respuesta depende del criterio de decisión. Si se

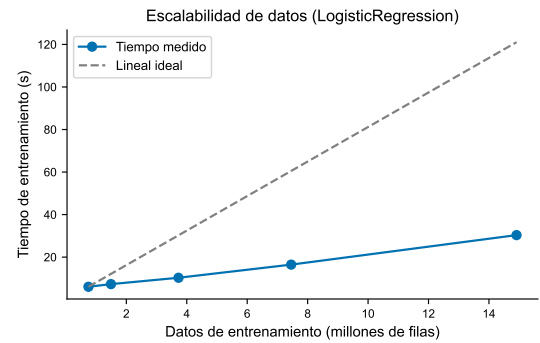


Figura 6. Escalabilidad de datos: el tiempo de entrenamiento crece por debajo de la referencia lineal al aumentar el volumen.

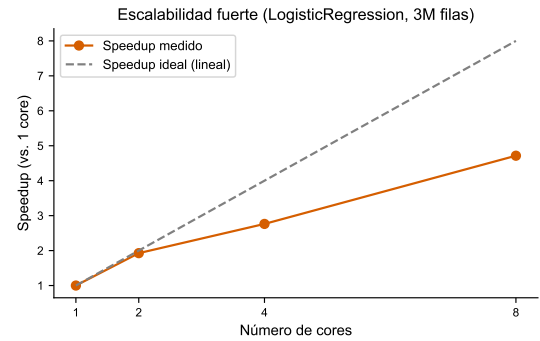


Figura 7. Escalabilidad fuerte: *speedup* frente al número de núcleos. La separación creciente respecto a la recta ideal refleja la ley de Amdahl.

prioriza el desempeño predictivo, el gradient-boosted trees es la mejor opción, con el mayor AUC y el mayor recall de la minoría, aunque al mayor costo de cómputo. Si se incorpora el costo, el random forest emerge como el compromiso más equilibrado: un AUC estadísticamente indistinguible del GBT (diferencia de 0,006 con un único particionado, sin prueba de significancia) en la mitad del tiempo. Si el costo es la restricción dominante, la regresión logística ofrece un AUC competitivo a una quinta parte del tiempo del GBT. La lección metodológica central, sin embargo, es transversal a la elección del modelo: la exactitud no es una métrica honesta bajo desbalance. Un clasificador trivial alcanzaría un 90,6 % de exactitud sin detectar un solo viaje de propina baja; los modelos resultan útiles solo cuando se observan el AUC y el recall de la clase minoritaria y se aplica ponderación de clases [6], [13].

El desempeño absoluto es moderado (AUC en torno a 0,68), lo que indica que la propina es difícil de predecir a partir de las variables operativas del viaje. Esto es coherente con la naturaleza del fenómeno: la propina depende de factores que el registro administrativo no captura, como la interacción con el conductor, la satisfacción del pasajero o las preferencias individuales. La señal disponible es real pero limitada, y ningún algoritmo puede extraer información que los datos no contienen.

Desde la perspectiva de las cinco V, el trabajo ejercita el *volumen* (decenas de millones de registros), la *variedad* (atributos espaciales, temporales y económicos), la *veracidad* (la decisión de restringirse a pagos con tarjeta para asegurar la fiabilidad de la propina) y el *valor* (anticipar la recompensa del conductor). La escalabilidad observada confirma que la arquitectura es apropiada para el crecimiento del volumen.

VII. LIMITACIONES

El estudio presenta limitaciones que delimitan la validez de sus conclusiones. Primero, la restricción a pagos con tarjeta introduce un sesgo de selección: el comportamiento de propina en efectivo, no registrado, podría diferir. Segundo, la comparación se basa en un único particionado entrenamiento/prueba con una semilla fija; no se calcularon intervalos de confianza ni pruebas de significancia, por lo que las diferencias pequeñas de AUC entre los mejores modelos (del orden de 0,006) deben interpretarse como equivalencia práctica y no como superioridad. Tercero, la ejecución en modo local no captura los costos de red y *shuffle* entre nodos de un clúster real, por lo que las curvas de escalabilidad deben leerse como una cota optimista. Cuarto, no se realizó una búsqueda exhaustiva de hiperparámetros, de modo que los desempeños reportados son representativos pero no necesariamente óptimos.

VIII. CONCLUSIONES

Se comparó de forma controlada el desempeño y el costo de cuatro clasificadores distribuidos de Spark MLlib para predecir la propina baja en viajes de taxi de Nueva York, sobre 14,9 millones de registros y bajo un desbalance de 9,7:1. La experiencia con el análisis de los datos confirma tres aprendizajes. Primero, en problemas desbalanceados la elección del modelo debe guiarse por el AUC y el recall de la minoría, y no por la exactitud, que un clasificador trivial inflaría hasta el 90,6 % sin utilidad alguna. Segundo, no existe un único modelo dominante: el GBT maximizó el desempeño, el random forest ofreció el mejor equilibrio desempeño-costo y la regresión logística fue la más eficiente, de modo que la decisión depende del presupuesto de cómputo. Tercero, los experimentos de escalabilidad confirmaron empíricamente las ventajas del procesamiento distribuido y sus límites teóricos según la ley de Amdahl. Como trabajo futuro se plantea incorporar técnicas de remuestreo distribuido y aprendizaje sensible al costo, enriquecer las variables con datos contextuales (clima, eventos, tráfico) y desplegar el *pipeline* en un clúster real para validar la escalabilidad horizontal a escala de múltiples nodos.

REFERENCIAS

- [1] A. Gandomi and M. Haider, "Beyond the hype: Big data concepts, methods, and analytics," *International Journal of Information Management*, vol. 35, no. 2, pp. 137–144, 2015.
- [2] M. Zaharia, R. S. Xin, P. Wendell, T. Das, M. Armbrust, A. Dave, X. Meng, J. Rosen, S. Venkataraman, M. J. Franklin, A. Ghodsi, J. Gonzalez, S. Shenker, and I. Stoica, "Apache spark: A unified engine for big data processing," *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016.
- [3] X. Meng, J. Bradley, B. Yavuz, E. Sparks, S. Venkataraman, D. Liu, J. Freeman, D. Tsai, M. Amde, S. Owen, D. Xin, R. Xin, M. J. Franklin, R. Zadeh, M. Zaharia, and A. Talwalkar, "MLlib: Machine learning in apache spark," *Journal of Machine Learning Research*, vol. 17, no. 34, pp. 1–7, 2016.
- [4] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," *Communications of the ACM*, vol. 51, no. 1, pp. 107–113, 2008.
- [5] S. Landset, T. M. Khoshgoftaar, A. N. Richter, and T. Hasanin, "A survey of open source tools for machine learning with big data in the Hadoop ecosystem," *Journal of Big Data*, vol. 2, no. 1, p. 24, 2015.
- [6] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, 2009.
- [7] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- [8] C. Elkan, "The foundations of cost-sensitive learning," in *Proc. 17th International Joint Conference on Artificial Intelligence (IJCAI)*, vol. 2, pp. 973–978, 2001.
- [9] A. Fernández, S. del Río, N. V. Chawla, and F. Herrera, "An insight into imbalanced big data classification: Outcomes and challenges," *Complex & Intelligent Systems*, vol. 3, no. 2, pp. 105–120, 2017.
- [10] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [11] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.
- [12] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. on Knowledge Discovery and Data Mining (KDD)*, pp. 785–794, 2016.
- [13] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.
- [14] X. Yu, "Big data driven model for new york taxi trips analysis," in *Proc. IEEE 6th Int. Conf. on Big Data Analytics (ICBDA)*, 2021.
- [15] M. Poongodi, M. Malviya, C. Kumar, M. Hamdi, V. Vijayakumar, J. Nebhen, and H. Alyamani, "New york city taxi trip duration prediction using MLP and XGBoost," *International Journal of System Assurance Engineering and Management*, vol. 13, no. 1, pp. 16–27, 2022.
- [16] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," in *Proc. AFIPS Spring Joint Computer Conference*, pp. 483–485, 1967.